l

INSTITUTO TECNOLÓGICO DE MÉXICO

CAMPUS ORIZABA

“ACTIVIDAD 2”

EQUIPO 9

GARCIA CEDEÑO IRVING EMANUEL

ROJAS BONILLA SANTIAGO

MARTINEZ ROSAS JOSE ANGEL

MATERIA: PROGRAMACION ORIENTADA A OBJETOS

GRUPO: VESPERTINO 1-2 PM

MAESTRO: RAFAEL HERRERA GARCIA

FECHA DE ENTREGA: 4 DE MARZO DE 2026

Problema 1:

En cierta empresa de fabricación de ropa, se requiere sistematizar el proceso, se fabrican diversidad de prendas de vestir tanto para caballeros como damas, no se fabrica ropa de niños, de cada prenda se tiene un registro de modelo, tela, costo de producción por pieza, genero (masculino, femenino o mixto), temporada (primavera, verano, otoño o invierno), cuando se produce un lote de cada prenda, se registra número de lote, número de piezas, fecha de fabricación. De cada lote debe calcularse el costo de producción del lote, así como el monto de recuperación que se obtendrá después de su venta, el precio de venta por pieza es siempre 15% más del costo de producción y por lote completo es por cada pieza el 5% más del costo de producción.

Análisis Del Problema:

En este problema vemos que es sobre una fábrica de ropa, lo cual lo dividimos en dos clases:

Clase Prenda:

La clase Prenda representa cada tipo de prenda que fabrica la empresa. En esta clase se guardan datos como el modelo, la tela utilizada, el costo de producción por pieza, el género y la temporada. Una prenda puede tener uno o varios lotes de producción, ya que cada vez que se fabrica se registra un nuevo lote.

Además, la clase permite calcular el precio de venta por pieza, el cual se obtiene aumentando el 15% al costo de producción.

Clase Lote:

La clase Lote representa cada lote de producción de una prenda. Cada lote tiene su número de lote, el número de piezas producidas y la fecha de fabricación.

A partir de esta información se puede calcular el costo total de producción del lote y el monto de recuperación después de la venta.

Clase prenda:

Prenda			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
modelo	String	1	si
tela	String	1	si
costoProduccionPZ	Double	1	si
genero	String	1	si
temporada	String	1	si

Operaciones:

Prenda			
operación	Tipo de dato(retorno)	argumentos	Alcance
constructor		(Modelo, tela, genero, temporada)	Public
calcularPrecioVenta	double		public

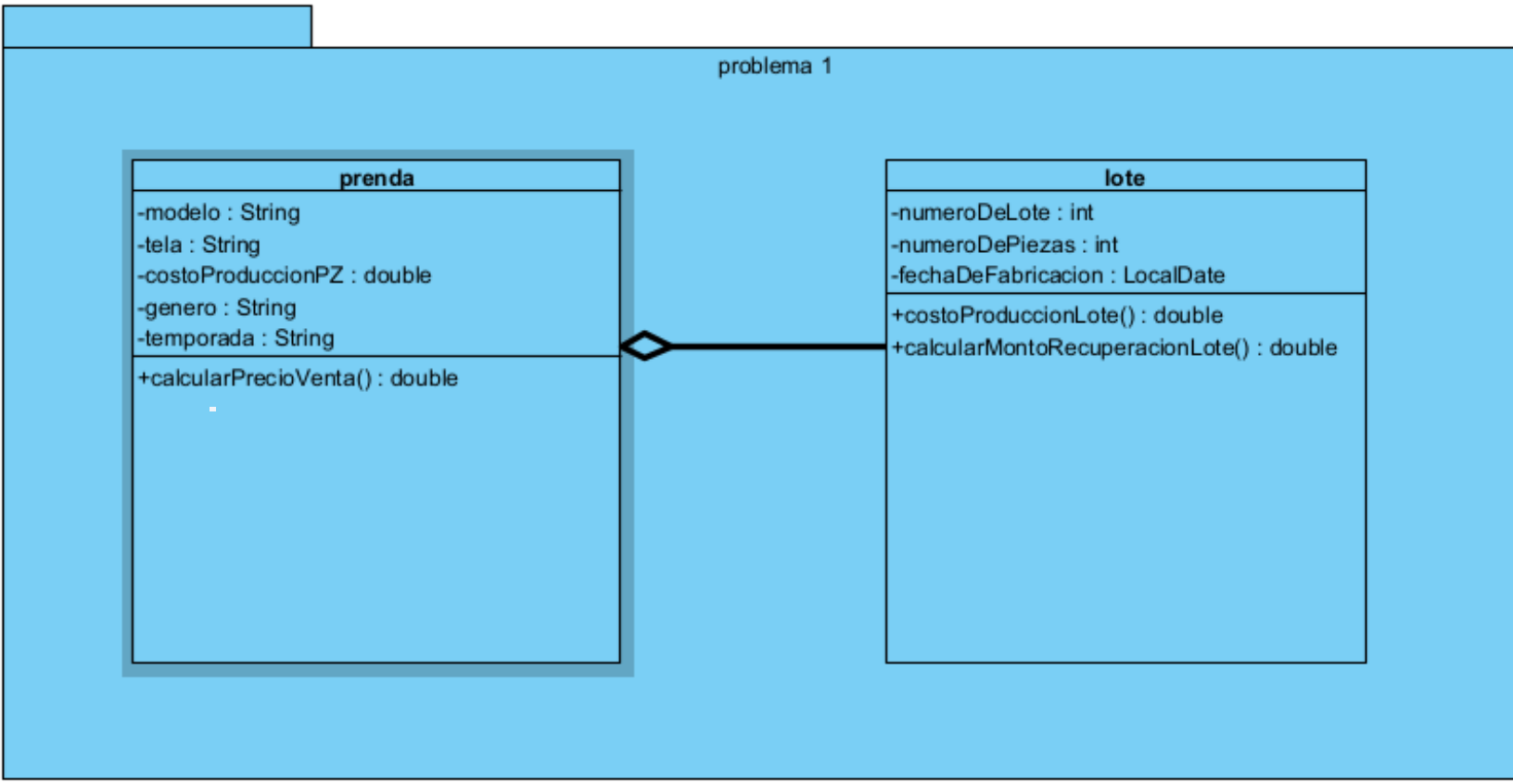
Clase lote:

Lote			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
numeroDeLote	Int	1	Si
númeroDePiezas	int	1	Si
fechaDeFabricación	LocalDate	1	Si

Operaciones:

Lote			
operación	Tipo de dato(retorno)	argumentos	Alcance
constructor		(numeroDeLote, numeroDePiezas, fechaDeFabricacion)	
costoProduccionLote	double		public
calcularMontoRecuperacionLote	double		public

Diagrama:


problema 1
prenda
-modelo : String
-tela : String
-costoProduccionPZ : double
-genero : String
-temporada : String
+calcularPrecioVenta() : double
lote
-numeroDeLote : int
-numeroDePiezas : int
-fechaDeFabricacion : LocalDate
+costoProduccionLote() : double
+calcularMontoRecuperacionLote() : double

Código clase prenda:

```java
package fabrica.ropa;

public class prenda {  no usages  Irving Emanuel Garcia Cedeño *
    private String modelo;  4 usages
    private String Tela;  4 usages
    private double precioventaPZ;  5 usages
    private double precioVentaLote;  4 usages
    private String genero;  4 usages
    private String temporada;  4 usages

    public prenda(String modelo, String tela, double precioventaPZ, String genero, String temporada) {
        this.modelo = modelo;
        Tela = tela;
        this.precioventaPZ = precioventaPZ;
        this.precioVentaLote = precioVentaLote;
        this.genero = genero;
        this.temporada = temporada;
    }

    public String getTela() {  no usages  new *
        return Tela;
    }

    public void setTela(String tela) {  no usages  new *
        Tela = tela;
    }
```

```java
public class prenda {  no usages  Irving Emanuel Garcia Cedeño *
    public String getModelo() {  no usages  new *
        return modelo;
    }

    public void setModelo(String modelo) {  no usages  new *
        this.modelo = modelo;
    }

    public double getPrecioventaPZ() {  no usages  new *
        return precioventaPZ;
    }

    public double getPrecioVentaLote() {  no usages  new *
        return precioVentaLote;
    }

    public String getGenero() {  no usages  new *
        return genero;
    }
```

prenda.java × lote.java

```java
public class prenda {  no usages  Irving Emanuel Garcia Cedeño *
    public String getTemporada() {  no usages  new *
        return temporada;
    }

    public void setTemporada(String temporada) {  no usages  new *
        this.temporada = temporada;
    }

    public double precioVentaPZ() {  no usages  Irving Emanuel Garcia Cedeño *
        return precioventaPZ*1.15;

    }
    public double precioventaLote(){  no usages  new *
        return precioventaPZ * 1.05;
    }

    @Override  new *
    public String toString() {
        return "prenda{" +
                "modelo='" + modelo + '\'' +
                ", Tela='" + Tela + '\'' +
                ", precioventaPZ=" + precioventaPZ +
                ", precioVentaLote=" + precioVentaLote +
                ", genero='" + genero + '\'' +
                ", temporada='" + temporada + '\'' +
```

prenda.java × lote.java

```java
public class prenda {  no usages  Irving Emanuel Garcia Cedeño *
        return precioventaPZ*1.15;

    }
    public double precioventaLote(){  no usages  new *
        return precioventaPZ * 1.05;
    }

    @Override  new *
    public String toString() {
        return "prenda{" +
                "modelo='" + modelo + '\'' +
                ", Tela='" + Tela + '\'' +
                ", precioventaPZ=" + precioventaPZ +
                ", precioVentaLote=" + precioVentaLote +
                ", genero='" + genero + '\'' +
                ", temporada='" + temporada + '\'' +
                '}';
    }
}
```

Código clase lote:

```java
package fabrica.ropa;
import java.time.LocalDate;


public class lote {
    private int numDeLote;
    private int numDePIezas;
    private LocalDate fechadeFabricacion;
    private double costoProduccionPz;

    public lote(int numDeLote, int numDePIezas, LocalDate fechadeFabricacion, double costoProduccionPz) {
        this.numDeLote = numDeLote;
        this.numDePIezas = numDePIezas;
        this.fechadeFabricacion = fechadeFabricacion;
        this.costoProduccionPz = costoProduccionPz;
    }

    public int getNumDePIezas() {
        return numDePIezas;
    }

    public void setNumDePIezas(int numDePIezas) {
        this.numDePIezas = numDePIezas;
    }
```

```java
public class lote {
    public int getNumDeLote() {
        return numDeLote;
    }

    public void setNumDeLote(int numDeLote) {
        this.numDeLote = numDeLote;
    }

    public LocalDate getFechadeFabricacion() {
        return fechadeFabricacion;
    }

    public double getCostoProduccionPz() {
        return costoProduccionPz;
    }

    public void setCostoProduccionPz(double costoProduccionPz) {
        this.costoProduccionPz = costoProduccionPz;
    }
```

```java
public class lote {
    }

    public double costoProduccionLote(){
        return numDePIezas*costoProduccionPz;
    }
    public double calcularMontoRecuperacion(){
        double precioVenta=costoProduccionPz*1.15;
        return numDePIezas*precioVenta;
    }

    @Override
    public String toString() {
        return "lote{" +
                "numDeLote=" + numDeLote +
                ", numDePIezas=" + numDePIezas +
                ", fechadeFabricacion=" + fechadeFabricacion +
                '}';
    }
}
```

Prueba:

```
jshell> import fabrica.ropa.lote

jshell> import fabrica.ropa.prenda;

jshell> import java.time.LocalDate;

jshell> prenda p1= new prenda ("Playera","Algodon",120.0,"Hombre","Verano");
p1 ==> prenda{modelo='Playera', Tela='Algodon', preciove ... mbre', temporada='Verano'}

jshell> p1.getModelo();
$5 ==> "Playera"

jshell> p1.getTela();
$6 ==> "Algodon"

jshell> p1.getGenero();
$7 ==> "Hombre"

jshell> p1.getTemporada();
$8 ==> "Verano"

jshell> p1.getPrecioventaPZ();
$9 ==> 120.0

jshell> p1.getPrecioVentaLote();
$10 ==> 0.0
```

```
jshell>  lote l1 = new lote(101, 50, LocalDate.of(2024,5,10), 120.0);
l1 ==> lote{numDeLote=101, numDePIezas=50, fechadeFabricacion=2024-05-10}

jshell> l1.costoProduccionLote();
$14 ==> 6000.0

jshell> l1.calcularMontoRecuperacion();
$15 ==> 6900.0

jshell> System.out.println(p1);
prenda{modelo='Playera', Tela='Algodon', precioventaPZ=120.0, precioVentaLote=0.0, genero='Hombre', temporada='Verano'}

jshell> System.out.println(l1);
lote{numDeLote=101, numDePIezas=50, fechadeFabricacion=2024-05-10}
```

```
jshell> p1.precioVentaPZ();
$11 ==> 138.0

jshell> p1.precioventaLote();
$12 ==> 126.0
```

Análisis del código:

En esta parte del programa se creó un objeto llamado miPrenda, al cual se le asignaron los valores correspondientes a los atributos definidos en la clase *Prenda*, como el modelo, tipo de tela, costo de producción por pieza, género y temporada.

Posteriormente se imprimen los datos del objeto para verificar que la información se haya guardado correctamente.
También se utiliza el método calcularPrecioVenta(), el cual permite obtener el precio de venta por pieza aumentando el 15% al costo de producción, tal como lo indica el problema.

(Lote):

En esta parte se creó un objeto llamado miLote, al cual se le asignaron los valores definidos en la clase *Lote*, como el número de lote, el número de piezas y la fecha de fabricación.

Después se utilizan los métodos de la clase para realizar los cálculos solicitados en el problema.
El método costoProduccionLote() calcula el costo total de producción multiplicando el número de piezas por el costo de producción por pieza.

Por otro lado, el método calcularMontoRecuperacionLote() calcula el monto de recuperación del lote considerando el precio de venta por pieza, el cual incluye un aumento sobre el costo de producción.

Finalmente, se imprimen los resultados para comprobar que los cálculos se realizaron correctamente.

Problema 2:

Una empresa agrícola ha decidido sistematizar el control de la producción frutal que tiene, para lo cual de cada fruta se describe su nombre, la extensión del terreno (hectáreas) dedicadas a su cultivo y producción, el tiempo de cosecha (Cada fruta puede tener 1 o más periodos de cosecha), la cantidad estimada a cosechar por hectárea en cada periodo (cada periodo por condiciones de estación climática puede tener diferentes cantidades estimadas), costo de producción promedio por tonelada y el precio de venta promedio por tonelada. Por cada fruta se puede agregar o eliminar un periodo de cosecha, se puede obtener el costo total de un periodo, así como las ganancias estimadas en un periodo.

Análisis Del Problema:

Ahora para este problema fue una empresa agrícola que controla la producción de frutas y ocupamos dos clases que son:

Clase Fruta: La clase Fruta representa cada tipo de fruta que produce la empresa agrícola. En esta clase se almacenan datos como el nombre de la fruta, la cantidad de hectáreas destinadas a su cultivo, el costo de producción por tonelada y el precio de venta por tonelada.

Además, cada fruta puede tener uno o varios periodos de cosecha, por lo que se utiliza una lista para almacenar los diferentes periodos registrados.

Las operaciones de esta clase permiten agregar o eliminar periodos de cosecha para cada fruta.

Clase Periodo Cosecha: La clase PeriodoCosecha representa cada periodo en el que se realiza la cosecha de una fruta. Esta clase contiene el nombre del periodo y la cantidad estimada de toneladas que se pueden producir por hectárea.

También incluye operaciones que permiten calcular la producción total, el costo de producción y las ganancias estimadas durante ese periodo de cosecha.

Clases Fruta

fruta			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
nombreFruta	String	1	Si
hectáreas	Double	1	Si
costoProducciónToneladas	Double	1	Si
precioVentaToneladas	Double	1	Si
periodos	ArrayList<periodoCosecha>	0	si

Operaciones:

Fruta			
Operación	Tipo de dato (retorno)	Argumentos	Alcance
constructor		nombreFruta, hectáreas, costoProduccionToneladas, periodos	
agregarPeriodo	void	periodoCosecha, periodo	public
eliminarPeriodo	void	Periodo, periodoCosecha	public

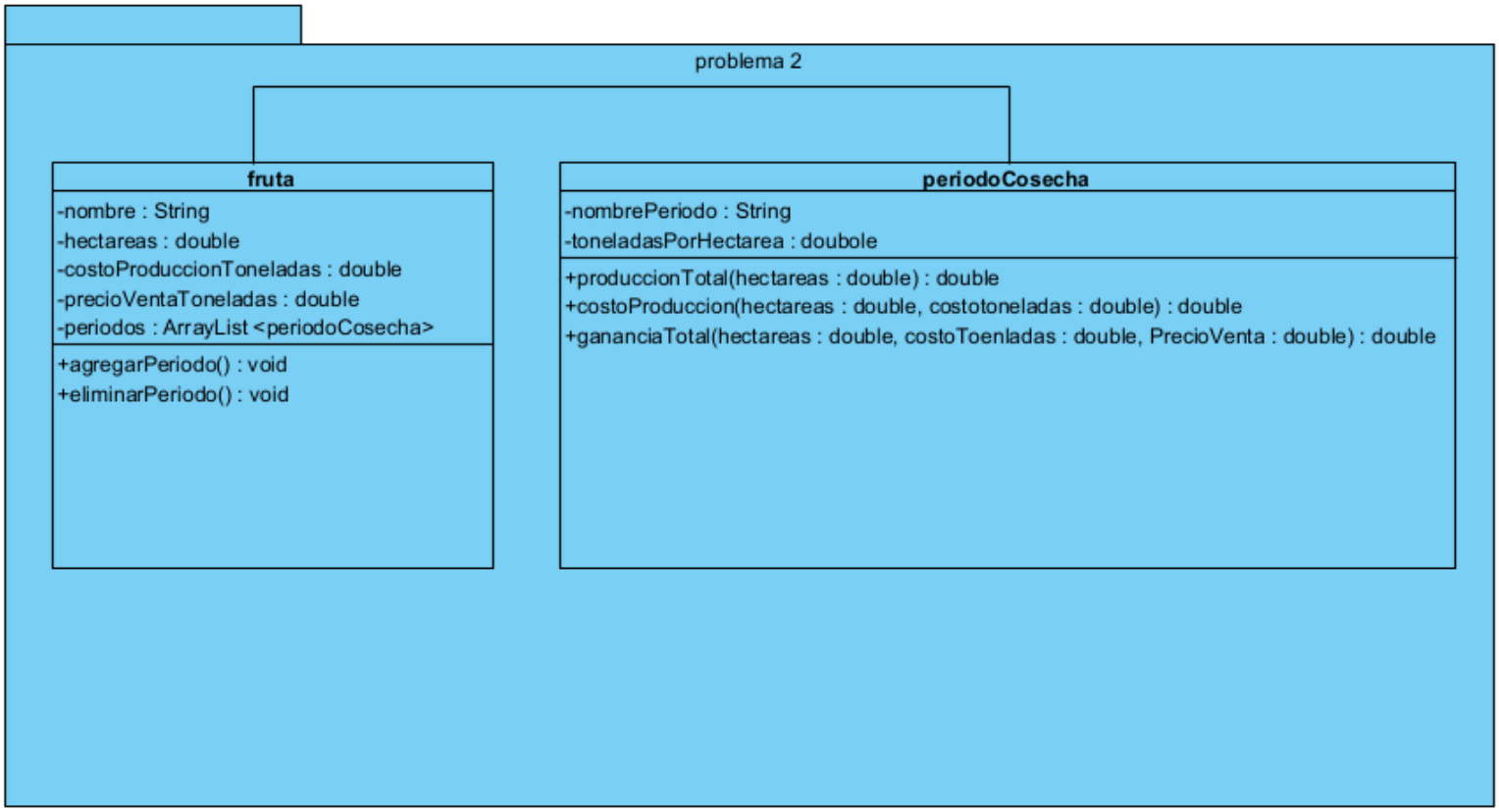
Clase Periodo Cosecha:

PeriodoCosecha			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
nombrePeriodo	String	1	Si
toneladasPorHectarea	Double	1	Si

Operaciones:

periodoCosecha			
Operación	Tipo de dato (retorno)	argumentos	Alcance
constructor		nombrePeriodo, toneladasPorHectarea	
produccionTotal	Double	hectáreas	Publico
costoProduccion	Double	Hectáreas, costoToneladas	Publico
gananciaTotal	Double	hectáreas, costoToneladas, precioVenta	Publico

Diagrama:



Código Clase Fruta:

fruta.java × periodoCosecha.java

```java
package empresa.agricola;
import java.util.ArrayList;


public class fruta {  no usages  Irving Emanuel Garcia Cedeño *
        private String nombreFruta;  4 usages
        private double hectareas;  5 usages
        private double costoProduccionTon;  5 usages
        private double precioVentaTon;  5 usages
        private ArrayList<periodoCosecha> periodos;  4 usages

        public fruta(String nombreFruta, double hectareas, double costoProduccionTon, double precioVentaTon){
            this.nombreFruta = nombreFruta;
            this.hectareas = hectareas;
            this.costoProduccionTon = costoProduccionTon;
            this.precioVentaTon = precioVentaTon;
            this.periodos = new ArrayList<>();
        }

    public String getNombreFruta() {  no usages  new *
        return nombreFruta;
    }

    public void setNombreFruta(String nombreFruta) {  no usages  new *
        this.nombreFruta = nombreFruta;
    }
```

fruta.java × periodoCosecha.java

```java
public class fruta {  no usages  Irving Emanuel Garcia Cedeño *
    public double getHectareas() {  no usages  new *
        return hectareas;
    }

    public void setHectareas(double hectareas) {  no usages  new *
        this.hectareas = hectareas;
    }

    public double getCostoProduccionTon() {  no usages  new *
        return costoProduccionTon;
    }

    public void setCostoProduccionTon(double costoProduccionTon) {  no usages  n
        this.costoProduccionTon = costoProduccionTon;
    }

    public double getPrecioVentaTon() {  no usages  new *
        return precioVentaTon;
    }

    public void setPrecioVentaTon(double precioVentaTon) {  no usages  new *
        this.precioVentaTon = precioVentaTon;
    }

    public ArrayList<periodoCosecha> getPeriodos() {  no usages  new *
        return periodos;
    }
```

```java
public class fruta {  no usages  Irving Emanuel Garcia Cedeño *

    public ArrayList<periodoCosecha> getPeriodos() {  no usages  new *
        return periodos;
    }

    public void agregarPeriodo(periodoCosecha periodo){  no usages  Irving Emanuel Garci
        periodos.add(periodo);
    }

    public void eliminarPeriodo(periodoCosecha periodo){  no usages  Irving Emanue
        periodos.remove(periodo);
    }
    public double gananciaPeriodo(periodoCosecha p){  no usages  new *
    return p.gananciaTotal(hectareas, costoProduccionTon, precioVentaTon);
    }

    @Override  Irving Emanuel Garcia Cedeño
    public String toString(){
        return "Fruta: " + nombreFruta +
                "\nHectareas: " + hectareas +
                "\nCosto Produccion Ton: " + costoProduccionTon +
                "\nPrecio Venta Ton: " + precioVentaTon;
    }
}
```

Codigo Clase PeriodoCosecha

```java
fruta.java    periodoCosecha.java

package empresa.agricola;

public class periodoCosecha {  5 usages  Irving Emanuel Garcia Cedeño *
    private String NombrePeriodo;  4 usages
    private double CantidadToneladas;  5 usages

    public periodoCosecha(String nombrePeriodo, double cantidadToneladas) {  no
        this.NombrePeriodo = nombrePeriodo;
        this.CantidadToneladas = cantidadToneladas;
    }

    public String getNombrePeriodo() {  no usages  new *
        return NombrePeriodo;
    }

    public void setNombrePeriodo(String nombrePeriodo) {  no usages  new *
        NombrePeriodo = nombrePeriodo;
    }

    public double getCantidadToneladas() {  no usages  new *
        return CantidadToneladas;
    }

    public void setCantidadToneladas(double cantidadToneladas) {  no usages  new *
        CantidadToneladas = cantidadToneladas;
    }
```

```java
fruta.java    periodoCosecha.java

public class periodoCosecha {  5 usages  Irving Emanuel Garcia Cedeño *

    public double ProduccionTotal(double Hectareas) {  2 usages  Irving Emanuel Garcia Cedeño
        return CantidadToneladas*Hectareas;

    }
    public double CostoProduccion(double Hectareas, double CostoTonleadas){  no usages  Irving Em
        return ProduccionTotal(Hectareas)*CostoTonleadas;
    }

    public double gananciaTotal(double Hectareas,double CostoToneladas, double precioVenta){
        double produccion=ProduccionTotal(Hectareas);
        return (produccion*precioVenta)- (produccion*CostoToneladas);
    }

    @Override  Irving Emanuel Garcia Cedeño
    public String toString() {
        return "PeriodoCosecha{" +
                "NombrePeriodo='" + NombrePeriodo + '\'' +
                ", CantidadToneladas=" + CantidadToneladas +
                '}';
    }
}
```

Prueba:

```
jshell> import empresa.agricola.fruta;

jshell> import empresa.agricola.periodoCosecha;

jshell> import java.util.ArrayList;

jshell> periodoCosecha p1 = new periodoCosecha("Primavera", 4.5);
p1 ==> PeriodoCosecha{NombrePeriodo='Primavera', CantidadToneladas=4.5}

jshell> periodoCosecha p2 = new periodoCosecha("Verano", 6.2);
p2 ==> PeriodoCosecha{NombrePeriodo='Verano', CantidadToneladas=6.2}

jshell> fruta f1 = new fruta("Mango", 10, 2000, 3500);
f1 ==> Fruta: Mango
Hectareas: 10.0
Costo Produccion Ton ... 0
Precio Venta Ton: 3500.0

jshell> f1.agregarPeriodo(p1);

jshell> f1.agregarPeriodo(p2);

jshell> f1.getNombreFruta();
$9 ==> "Mango"
```

Prueba

```
jshell> f1.getHectareas();
$10 ==> 10.0

jshell> f1.getCostoProduccionTon();
$11 ==> 2000.0
```

```
jshell> f1.getPrecioVentaTon();
$12 ==> 3500.0

jshell> p1.ProduccionTotal(f1.getHectareas());
$13 ==> 45.0

jshell> p1.CostoProduccion(f1.getHectareas(), f1.getCostoProduccionTon());
$14 ==> 90000.0
```

```
jshell> f1.gananciaPeriodo(p1);
$15 ==> 67500.0

jshell> System.out.println(p1);
PeriodoCosecha{NombrePeriodo='Primavera', CantidadToneladas=4.5}

jshell> System.out.println(f1);
Fruta: Mango
Hectareas: 10.0
Costo Produccion Ton: 2000.0
Precio Venta Ton: 3500.0
```

Análisis del código:

En esta parte del programa se crea un objeto llamado fruta1, al cual se le asignan los valores correspondientes a los atributos de la clase *Fruta*, como el nombre de la fruta, la cantidad de hectáreas, el costo de producción por tonelada y el precio de venta por tonelada.

Posteriormente se crea un objeto de la clase PeriodoCosecha, donde se especifica el nombre del periodo y la cantidad de toneladas estimadas por hectárea.

Después se utiliza el método agregarPeriodo() para agregar ese periodo de cosecha a la fruta.

En la clase *PeriodoCosecha* se utilizan diferentes métodos para realizar los cálculos solicitados.
El método produccionTotal() calcula la producción total multiplicando las toneladas por hectárea por el número de hectáreas.

El método costoProduccion() calcula el costo total multiplicando la producción total por el costo de producción por tonelada.

Finalmente, el método gananciaTotal() calcula las ganancias estimadas restando el costo total de producción al ingreso total obtenido por la venta de las toneladas producidas.

Problema 3:

El observatorio astronómico de San Pedro Mártir, de acuerdo a sus observaciones necesita un catálogo de los cuerpos celestes descubiertos para lo cual de cada cuerpo celeste se describe la ubicación en el espacio estelar en relación a las posiciones geográficas espaciales (expresado en grado de latitud (norte o sur) y grados de longitud (este o oeste)) periodo de ubicación (mes(es) donde se observó en dicha posición), distancia aproximada a la tierra (años luz o km/segundos), el nombre que se le asigno, la composición del cuerpo estelar (gases, solidos, líquidos). De cada cuerpo celeste se puede obtener el desplazamiento con respecto a otra observación del mismo cuerpo en un periodo distinto.

Análisis del problema:

Clase cuerpo celeste: La clase CuerpoCeleste representa cada cuerpo celeste registrado por el observatorio. En esta clase se guardan datos como el nombre del cuerpo celeste, su composición y la distancia aproximada a la Tierra junto con la unidad en la que se mide.

Además, esta clase incluye una operación que permite calcular el desplazamiento del cuerpo celeste comparando dos observaciones realizadas en distintos periodos.

Clase periodo visible: La clase PeriodoVisible representa el periodo en el que el cuerpo celeste fue observado en una determinada posición. En esta clase se guarda el nombre del periodo de observación y la ubicación espacial donde se encontraba el cuerpo celeste.

Clase Ubicación espacial: La clase UbicacionEspacial representa las coordenadas espaciales del cuerpo celeste. Estas coordenadas están formadas por la latitud, el hemisferio de latitud, la longitud y el hemisferio de longitud.

Esta clase incluye un método que permite calcular el desplazamiento entre dos ubicaciones espaciales diferentes.

Clases Cuerpos celestes

Cuerpos celestes			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
nombre	String	1	si
composición	String	1	si
distanciaTierra	Double	1	si
unidadTierra	String	1	si

Operaciones:

Cuerpos celestes			
Operación	Tipo de dato (retorno)	argumentos	Alcance
constructor		Nombre, composición, distanciaTierra, unidadTierra	
calcularDesplazamiento	Double	Dos observaciones, una primero y otra después	public

Clase periodo Visible:

Periodo Visible			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
periodo	String	1	si
ubicacion	ubicacionEspacial	1	si

Operaciones:

Periodo Visible			
Operación	Tipo de dato (retorno)	argumentos	Alcance
constructor		Periodo, ubicacion	
mostrarUbicacion	void		public

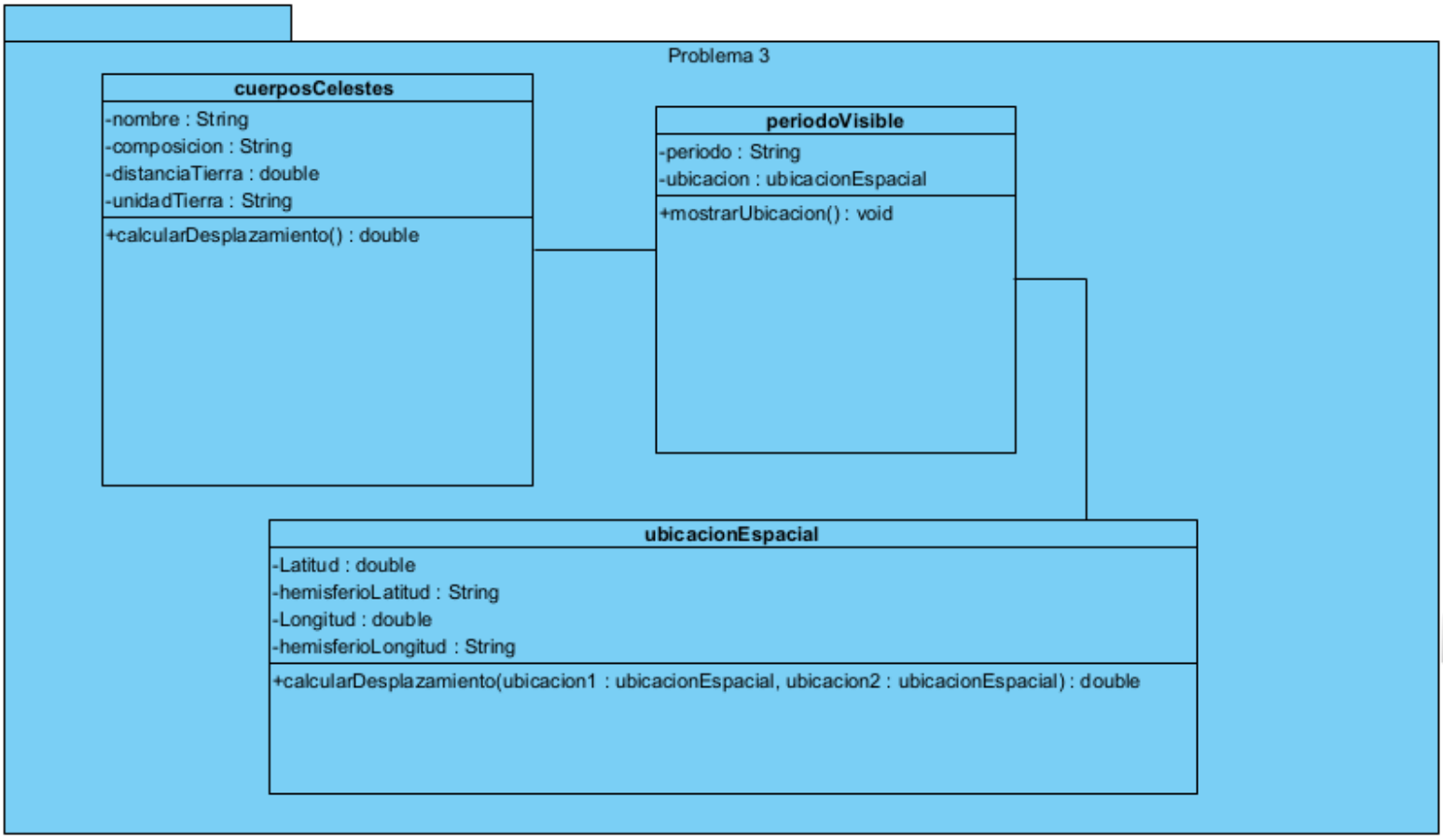
Clase Ubicación Especial

Ubicación Espacial			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
latitud	Double	1	Si
hemisferioLatitud	String	1	Si
longitud	Double	1	Si
hemisferioLongitud	String	1	Si

Operaciones:

Ubicación Espacial			
Operación	Tipo de dato (retorno)	argumentos	Alcance
constructor		lataitud, hemisferioLatitud, longitud, hemisferioLongitud	
calcularDesplazamiento	Double	Ubicacion1, ubicacion2	Public

Diagrama:



Código Clase CuerposCelestes:

CuerposCelestes.java × UbicacionEspacial.java PeriodoVisible.java

```java
package observatorio;
import java.util.ArrayList;

public class CuerposCelestes {  no usages  Irving Emanuel Garcia Cedeño *
    private String Nombre;  4 usages
    private String  Composicion;  4 usages
    private double DistanciaTierra;  4 usages
    private String UnidadTierra;  4 usages
    private ArrayList <PeriodoVisible> Periodos;  4 usages

    public CuerposCelestes(String nombre, String composicion, double distanciaTierra, String unidadTierra, ArrayList<PeriodoVisible> periodos) {
        Nombre = nombre;
        Composicion = composicion;
        DistanciaTierra = distanciaTierra;
        UnidadTierra = unidadTierra;
        Periodos = periodos;
    }

    public String getNombre() {  no usages  new *
        return Nombre;
    }

    public void setNombre(String nombre) {  no usages  new *
        Nombre = nombre;
    }
```

CuerposCelestes.java × UbicacionEspacial.java PeriodoVisible.java

```java
4   public class CuerposCelestes {  no usages  Irving Emanuel Garcia Cedeñ
27      public String getComposicion() {  no usages  new *
28          return Composicion;
29      }
30
31      public void setComposicion(String composicion) {  no usages
32          Composicion = composicion;
33      }
34
35      public double getDistanciaTierra() {  no usages  new *
36          return DistanciaTierra;
37      }
38
39      public void setDistanciaTierra(double distanciaTierra) {
40          DistanciaTierra = distanciaTierra;
41      }
42
43      public String getUnidadTierra() {  no usages  new *
44          return UnidadTierra;
45      }
46
47      public void setUnidadTierra(String unidadTierra) {  no usage
48          UnidadTierra = unidadTierra;
49      }
50
51      public ArrayList<PeriodoVisible> getPeriodos() {  no usages
52          return Periodos;
53      }
```

CuerposCelestes.java × UbicacionEspacial.java PeriodoVisible.java

```java
4   public class CuerposCelestes {  no usages  Irving Emanuel Garcia Cede
52          return Periodos;
53      }
54
55      public void agregarPeriodo(PeriodoVisible p){  no usages  ne
56          Periodos.add(p);
57      }
58      public void eliminarPeriodo(PeriodoVisible p){  no usages  n
59          Periodos.remove(p);
60      }
61
62      @Override  Irving Emanuel Garcia Cedeño *
63      public String toString() {
64          return "cuerposCelestes{" +
65                  "Nombre='" + Nombre + '\'' +
66                  ", Composicion='" + Composicion + '\'' +
67                  ", DistanciaTierra=" + DistanciaTierra +
68                  ", UnidadTierra='" + UnidadTierra + '\'' +
69                  '}';
70      }
71  }
```

Código clase ubcacionEspacial:

```java
package observatorio;

public class UbicacionEspacial {
    private double Latitud;
    private String HemisferioLatitud;
    private double Longitud;
    private String HemisferioLongitud;

    public UbicacionEspacial(double latitud, String hemisferioLatitud, String hemisferioLongitud, double longitud) {
        this.Latitud = latitud;
        this.HemisferioLatitud = hemisferioLatitud;
        this.HemisferioLongitud = hemisferioLongitud;
        this.Longitud = longitud;
    }

    public double getLatitud() {
        return Latitud;
    }

    public void setLatitud(double latitud) {
        Latitud = latitud;
    }

    public String getHemisferioLatitud() {
        return HemisferioLatitud;
    }
```

```java
public class UbicacionEspacial {
    public void setHemisferioLatitud(String hemisferioLatitud) {
        HemisferioLatitud = hemisferioLatitud;
    }

    public double getLongitud() {
        return Longitud;
    }

    public void setLongitud(double longitud) {
        Longitud = longitud;
    }

    public String getHemisferioLongitud() {
        return HemisferioLongitud;
    }

    public void setHemisferioLongitud(String hemisferioLongitud) {
        HemisferioLongitud = hemisferioLongitud;
    }

    public double calcularDesplazamiento(UbicacionEspacial otra){
        double dx = this.Latitud - otra.Latitud;
        double dy = this.Longitud - otra.Longitud;
```

```java
public class UbicacionEspacial {  5 usages  Irving Emanuel Garcia Cedeño *
        return HemisferioLongitud;
    }

    public void setHemisferioLongitud(String hemisferioLongitud) {
        HemisferioLongitud = hemisferioLongitud;
    }

    public double calcularDesplazamiento(UbicacionEspacial otra){
        double dx = this.Latitud - otra.Latitud;
        double dy = this.Longitud - otra.Longitud;
        return Math.sqrt(dx*dx + dy*dy);
    }

    @Override  Irving Emanuel Garcia Cedeño *
    public String toString() {
        return "ubicacionEspacial{" +
                "longitud=" + Longitud +
                ", latitud=" + Latitud +
                '}';
    }
}
```

Código clase periodoVisible:

```java
package observatorio;

public class PeriodoVisible {  5 usages  Irving Emanuel Garcia Cedeño *
    private String Periodo;  4 usages
    private UbicacionEspacial Ubicacion;  4 usages

    public PeriodoVisible(String periodo, UbicacionEspacial ubicacion) {
        Periodo = periodo;
        Ubicacion = ubicacion;
    }

    public String getPeriodo() {  no usages  new *
        return Periodo;
    }

    public void setPeriodo(String periodo) {  no usages  new *
        Periodo = periodo;
    }

    public UbicacionEspacial getUbicacion() {  no usages  new *
        return Ubicacion;
    }

    public void setUbicacion(UbicacionEspacial ubicacion) {  no usages  new *
        Ubicacion = ubicacion;
    }
```

```java
© CuerposCelestes.java    © UbicacionEspacial.java    © PeriodoVisible.java ×

public class PeriodoVisible {  5 usages  Irving Emanuel Garcia Cedeño *

    public void setPeriodo(String periodo) {  no usages  new *
        Periodo = periodo;
    }

    public UbicacionEspacial getUbicacion() {  no usages  new *
        return Ubicacion;
    }

    public void setUbicacion(UbicacionEspacial ubicacion) {  no u
        Ubicacion = ubicacion;
    }

    @Override  Irving Emanuel Garcia Cedeño *
    public String toString() {
        return "periodoVisible{" +
                "periodo='" + Periodo + '\'' +
                ", ubicacion=" + Ubicacion +
                '}';
    }
}
```

Prueba:

```
jshell> import observatorio.cuerposCelestes;

jshell> import observatorio.PeriodoVisible;

jshell> import observatorio.ubicacionEspacial;

jshell> ubicacionEspacial u1 = new ubicacionEspacial(10.5, "N", 45.2, "E");
u1 ==> ubicacionEspacial{longitud=45.2, latitud=10.5}

jshell> ubicacionEspacial u2 = new ubicacionEspacial(12.0, "N", 47.8, "E");
u2 ==> ubicacionEspacial{longitud=47.8, latitud=12.0}

jshell> u1.calcularDesplazamiento(u2);
$6 ==> 3.001666203960722

jshell> PeriodoVisible p1 = new PeriodoVisible("Invierno", u1);
p1 ==> periodoVisible{periodo='Invierno', ubicacion=ubic ... gitud=45.2, latitud=10.5}}

jshell> PeriodoVisible p2 = new PeriodoVisible("Primavera", u2);
p2 ==> periodoVisible{periodo='Primavera', ubicacion=ubi ... gitud=47.8, latitud=12.0}}

jshell> cuerposCelestes c1 = new cuerposCelestes(
   ...>     "Marte",
   ...>     "Rocoso",
   ...>     225000000,
   ...>     "km",
   ...>     new java.util.ArrayList<>()
   ...> );
c1 ==> cuerposCelestes{nombre='Marte', composicion=Rocos ... a=2.25E8, unidadTierra=km}
```

Análisis del código:

En esta parte del programa primero se crea un objeto de la clase CuerposCelestes, donde se registran datos como el nombre del cuerpo celeste, su composición, la distancia aproximada a la Tierra y la unidad de medida.

Después se crea un objeto de la clase UbicacionEspacial, donde se especifican las coordenadas de latitud y longitud junto con sus respectivos hemisferios.

Posteriormente se utilizan los métodos de la clase para calcular el desplazamiento entre dos ubicaciones diferentes del mismo cuerpo celeste.

Finalmente, se imprimen los datos de los objetos creados para verificar que la información se haya almacenado correctamente y que los métodos funcionen adecuadamente.

CONCLUSION:

En esta actividad pudimos observar que tan importante es la abstracción y el encapsulamiento, porque hubo errores que cometimos al principio desde la actividad 1, y nos dimos cuenta de ciertos datos que nos hacían falta para poder darle solución a los problemas, como identificar que dato nos faltaba declarar dentro de la clase para poder realizar la operación necesaria, nos dimos cuenta como es que si no realizamos de manera efectiva y correcta la abstracción, al querer seguir con el desarrollo de nuestros problemas pues no podremos hacer mucho, porque nuestros "cimientos" o bases en la abstracción no son sólidas o claras, y es el principio básico de la programación orientada a objetos.

URl GIT:

https://github.com/soysantiagorb/actividadd2.corregida

https://github.com/irvingemanuel2003-a11y/actividadd2.git